Denoised Variance-Based Pruning with Optimal Brain Bias Compensation

Geon Tack Lee[1,2] **Jaegul Choo**[1,†] **Kang Eun Jeon**[1,†]
[1]DAVIAN Lab, KAIST AI [2]New York University Abu Dhabi
Email: gtl256@nyu.edu, jchoo@kaist.ac.kr, kejeon@kaist.ac.kr
[†]Corresponding authors.
August 19, 2026

Abstract

Vision Transformers (ViTs) achieve state-of-the-art performance but carry massive computational overhead that restricts edge deployment. Although structural pruning has emerged as a key strategy to reduce these costs, existing methods often suffer from severe accuracy degradation or require expensive retraining. Recently, Variance-Based Pruning (VBP) introduced a promising paradigm by selecting neurons based on activation variance; however, it remains limited by statistical noise in finite-sample activation covariance and reliance on bias-only updates that cannot fully account for structural reconstruction error. To address these limitations, we introduce Denoised Variance-Based Pruning with Optimal Brain Bias Compensation (DVBP + OB^{2}C). We leverage random matrix theory to filter noise from the activation covariance spectrum for robust neuron selection and mathematically prove that integrating mean-shift compensation into the Optimal Brain Compression objective reduces the layer-wise Hessian exactly to the activation covariance matrix. This enables an optimal, closed-form update of the remaining weights using the same statistics gathered for selection. Extensive experiments on DeiT, Swin, and ConvNeXt architectures demonstrate that DVBP + OB^{2}C achieves state-of-the-art training-free performance; at 50% MLP pruning, it retains over 90% of the original Top-1 accuracy on Small and Base variants, outperforming VBP by up to 29.46% (ConvNeXt-T) and 7.33% (Swin-S). The code is available at: https://github.com/geontackee/DVBP_OB2C.

Keywords: Vision Transformers, Structured Pruning, Variance-Based Pruning

1. Introduction

Since the introduction of the Vision Transformer (ViT) [6], transformer-based architectures have consistently defined the state-of-the-art in computer vision, frequently outperforming traditional Convolutional Neural Networks (CNNs) [16]. However, this empirical success comes at a steep computational price. Standard ViTs demand substantial memory and computational resources for inference [32], effectively precluding their deployment on resource-constrained edge devices. For instance, ViT-B/16 requires 86 million parameters [6], contrasting sharply with the 7.5 million of MobileNetV3 [18]. As vision models are increasingly deployed on-device for latency-critical applications, such as autonomous driving, robotics, and unmanned aerial systems, achieving smaller model footprints and higher inference throughput has become an indispensable research objective.

While pruning has emerged as a principled strategy to bridge this efficiency gap [14, 17], current paradigms present a fundamental trade-off between theoretical sparsity and practical acceleration. Unstructured methods [14, 21] achieve high compression ratios by zeroing individual weights. However, these methods produce irregular sparsity patterns that demand specialized hardware accelerators [7] or custom CUDA kernels to induce practical latency reductions [12]. This severely limits their scalability and real-world adoption. On the other hand, structured pruning [22, 35] physically removes entire neurons or filters, hence directly reducing model size and inference time on commodity hardware. However, existing structured methods predominantly follow the magnitude-based pruning paradigm [14, 22], which ranks parameters by the absolute magnitude of their weights under the assumption that smaller weights contribute less to the network. While this heuristic is effective for zeroing individual connections, removing entire neurons or

filters based on aggregate magnitude discards complex feature interactions and leads to severely degraded accuracy [25].

In a fundamentally different direction, *Variance-Based Pruning* (VBP) [2] has demonstrated that this trade-off need not be inevitable. Rather than relying on weight magnitudes, VBP introduces an activation variance criterion that identifies neurons whose activations exhibit the lowest variance across a calibration set. The key insight is that low-variance neurons behave approximately as constants, meaning their contribution to the downstream layer can be effectively absorbed into the bias term through a simple mean-shift compensation. This allows VBP to structurally remove neurons while immediately recovering a portion of the lost performance, substantially outperforming magnitude-based approaches. However, VBP still suffers from significant accuracy degradation at higher pruning ratios, leaving considerable room for improvement.

In this work, we identify two critical limitations of VBP. First, the activation covariance matrix, from which variance scores are derived, is corrupted by statistical noise. This noise obscures the true importance ranking of neurons, leading to increasingly suboptimal pruning decisions at aggressive compression ratios (e.g., ≥50%). Second, VBP compensates for pruned neurons solely through mean-shift compensation, leaving the remaining weight matrices entirely unmodified. We observe that mean-shift compensation cannot fully account for the reconstruction error introduced by removing entire neurons, and a principled compensation through weight update is necessary to close this gap.

To address these limitations, we propose Denoised Variance-Based Pruning with Optimal Brain Bias Compensation (DVBP + OB^{2}C). Our core contributions are:

- **Denoised Variance Metric:** We leverage random matrix theory, specifically the Marchenko-Pastur distribution, to separate signal eigenvalues from noise in the activation covariance spectrum. The resulting denoised variance scores yield a substantially more reliable neuron importance ranking, particularly at high pruning ratios.
- **OB^{2}C Framework:** We incorporate mean-shift compensation directly into the seminal Optimal Brain Compression (OBC) [10] reconstruction objective. This integration yields a remarkably elegant result: the layer-wise Hessian reduces exactly to the activation covariance matrix. Consequently, the same covariance matrix simultaneously serves as the basis for both neuron selection (via denoised variance) and optimal multi-weight recovery, unifying the two stages of our pipeline into a single, coherent statistical framework.

Extensive experiments on DeiT [32], Swin [23], and ConvNeXt [24] architectures across Tiny, Small, and Base scales demonstrate that our method consistently outperforms VBP by a widening margin as the pruning ratio increases. At a 50% structural reduction, DVBP + OB^{2}C retains approximately 80% Top-1 accuracy on Tiny-scale models and exceeds 90% on Small and Base variants, surpassing VBP by up to 7.33% on Swin-S and 29.46% on ConvNeXt-T.

2. Background and Related Works

2.1. Model Compression and Pruning

Model compression relies primarily on quantization [1, 11] and pruning [17, 20]. Unlike quantization, which reduces bit-width but often demands specialized low-bit hardware, pruning directly eliminates operations, natively accelerating inference. Pruning roots trace to second-order methods like OBD [19] and OBS [15]. Later approaches introduced magnitude thresholding [14], gradient-based sensitivities [21], and identified sparse, trainable subnetworks [8]. However, these *unstructured* methods create irregular sparsity requiring custom accelerators [12]. *Structured* pruning instead removes entire neurons or channels [22], yielding immediate speedups on commodity hardware at the cost of severe accuracy drops, typically requiring extensive iterative retraining [34].

2.2. Variance-Based Pruning

Variance-Based Pruning (VBP) [2] selects neurons for removal based on activation variance rather than weight magnitude. Given an input activation vector $\mathbf{x}$ with mean $\boldsymbol{\mu}$, VBP introduces a mean-shift vector to compensate for pruned neurons. The mean-shift vector $\Delta\boldsymbol{\mu}$'s j-th entry is $\boldsymbol{\mu}_j$ if neuron j is pruned and zero otherwise. The output of the pruned layer is then expressed as:

$$\mathbf{y} = \hat{\mathbf{W}}\hat{\mathbf{x}} + \mathbf{b}' \tag{1}$$

where $\hat{\mathbf{W}}$ is the pruned weight of the subsequent layer, $\hat{\mathbf{x}}$ is the pruned output, and $\mathbf{b}'$ is the mean-shift compensation term that absorbs the effect of the pruned neurons into the bias:

$$\mathbf{b}' = \mathbf{b} + \mathbf{W}\Delta\boldsymbol{\mu}. \tag{2}$$

After this compensation, the residual reconstruction error for neuron i is governed by its activation variance, σ_i^2:

$$\mathbb{E}[(x_i - \mathbb{E}[x_i])^2] = \mathbb{E}[(x_i - \mu_i)^2] = \sigma_i^2 \tag{3}$$

Therefore, pruning neurons with the lowest σ_i^2 minimizes the expected reconstruction error under bias compensation.

2.3. Optimal Brain Compression

The Optimal Brain Compression (OBC) framework [10] has served as the foundation for numerous model compression algorithms, notably extending to Large Language Models through methods such as GPTQ [11] and SparseGPT [9]. Building upon the Optimal Brain Surgeon (OBS) [15] framework, OBC minimizes the squared reconstruction error between the original and compressed layer outputs:

$$\underset{\hat{\mathbf{W}}}{\operatorname{argmin}} ||\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}\mathbf{X}||_F^2, \tag{4}$$

where $\hat{\mathbf{W}}$ is the compressed weight matrix and $\mathbf{X} \in \mathbb{R}^{d\times N}$ is the input activation matrix whose columns are activation vectors collected from N calibration samples. To minimize this objective, the OBC framework derives a block-wise closed-form update rule that adjusts the remaining weights as

$$\delta_P = -\mathbf{H}^{-1}_{:,P}((\mathbf{H}^{-1})_P)^{-1}\mathbf{w}_P \tag{5}$$

where δ_P is the weight update for block P, $\mathbf{H} = 2\mathbf{X}\mathbf{X}^T$ is the layer-wise proxy-Hessian, $\mathbf{w}_P$ is the current weight vector of the row being updated, and P denotes the set of indices corresponding to the block being compressed. Recent work OBS-Diff [36] adapts second-order statistics for structured pruning of diffusion models. However, OBS-Diff predominantly utilizes complex, Hessian-based OBS calculations as the primary criterion for selecting which structures to remove. Our method decouples this pipeline: we employ an efficient denoised variance metric to identify redundant neurons, reserving the OBC framework strictly for optimal weight recovery after pruning.

3. Methodology

In this section, we present the DVBP + OB^{2}C structured pruning pipeline. We begin by augmenting the OBC [10] reconstruction objective with mean-shift compensation (§3.1), showing that the resulting layer-wise Hessian reduces to the input activation covariance matrix. We then describe an online method for computing this covariance without prohibitive memory costs (§3.2). To address the inherent noise in finite-sample covariance estimates, we apply spectral denoising via the Marchenko-Pastur distribution (§3.3), yielding robust variance scores for neuron selection (§3.4). Finally, we detail the end-to-end pruning and weight update mechanism (§3.5).

3.1. Optimal Brain Bias Compensation (OB^{2}C)

To mitigate the mean-shift error introduced during weight pruning, we introduce Optimal Brain Bias Compensation (OB^{2}C), which explicitly incorporates a bias term into the layer-wise reconstruction objective. Given an input activation matrix $\mathbf{X} \in \mathbb{R}^{d\times N}$, original weights $\mathbf{W}$, pruned weights $\hat{\mathbf{W}}$, and a bias vector $\mathbf{b}$, the augmented objective is formulated as:

$$\arg\min_{\hat{\mathbf{W}},\mathbf{b}} \left\|\mathbf{W}\mathbf{X} - (\hat{\mathbf{W}}\mathbf{X} + \mathbf{b}\mathbf{1}^T)\right\|_F^2 \tag{6}$$

where $\mathbf{1}$ is an N-dimensional vector of ones. Setting the partial derivative of the objective with respect to $\mathbf{b}$ to zero yields the optimal bias compensation:

$$\mathbf{b}^* = (\mathbf{W} - \hat{\mathbf{W}})\boldsymbol{\mu} \tag{7}$$

where $\boldsymbol{\mu} = \frac{1}{N}\mathbf{X}\mathbf{1}$ denotes the mean vector of the input activations. Substituting $\mathbf{b}^*$ back into the objective function yields a simplified minimization problem:

$$\arg\min_{\Delta\mathbf{W}} \left\|\Delta\mathbf{W}\widetilde{\mathbf{X}}\right\|_F^2 \tag{8}$$

where $\Delta\mathbf{W} = \mathbf{W} - \hat{\mathbf{W}}$ represents the weight perturbation, and $\widetilde{\mathbf{X}} = \mathbf{X} - \boldsymbol{\mu}\mathbf{1}^T$ represents the mean-centered activations. For detailed mathematical derivation, see Appendix A.2.

Evaluating the error induced by this weight perturbation, we find that the layer-wise proxy-Hessian matrix with respect to the rows of $\Delta\mathbf{W}$ is given by:

$$\mathbf{H} = 2\widetilde{\mathbf{X}}\widetilde{\mathbf{X}}^\top = 2N\mathbf{C} \tag{9}$$

where N is the calibration set size and $\mathbf{C}$ is the activation covariance matrix. Consequently, the layer-wise Hessian is directly proportional to the covariance of the centered activations, rather than the uncentered second moment used in standard formulations. Integrating this into the OBC multi-weight update, we replace the standard Hessian with $2N\mathbf{C}$. Let P denote the set of indices corresponding to the pruned neurons. The update formula for the weights is rewritten as:

$$\delta_{\mathbf{W}} = -\mathbf{W}_{:,P}\left([\mathbf{C}^{-1}]_{P,P}\right)^{-1}[\mathbf{C}^{-1}]_{P,:} \tag{10}$$

This yields an update expression that is structurally identical to the standard OBC rule, but inherently minimizes the mean-shift error by operating on the centered covariance matrix $\mathbf{C}$.

3.2. Covariance Matrix Computation

To apply OB^{2}C, we first require the exact centered covariance matrix of the activations for each layer. Storing full-precision activations for a naive computation demands an $O(N)$ memory footprint, rendering the approach computationally infeasible on large calibration sets. To ensure our approach remains computationally tractable, we track each layer's covariance matrix in an online, batched manner during calibration. To track the activation statistics of neurons, the VBP [2] paper employs Welford's algorithm to iteratively update their mean and variance. In a similar vein, Chan et al. [3] introduced a robust formula for updating the mean and second central moment across partitioned dataset $\mathbf{A}$ partitioned into $\mathbf{B}$ and $\mathbf{C}$ where $\mathbf{A} = \mathbf{B} \cup \mathbf{C}$, using the expressions from Pébay [28]:

$$\boldsymbol{\mu}_A = \boldsymbol{\mu}_\mathbf{B} + n_\mathbf{C}\frac{\delta_{\mathbf{C},\mathbf{B}}}{n}, \qquad \mathbf{M}_{2,\mathbf{A}} = \mathbf{M}_{2,\mathbf{B}} + \mathbf{M}_{2,\mathbf{C}} + n_\mathbf{B}n_\mathbf{C}\frac{\delta_{\mathbf{C},\mathbf{B}}^2}{n_A} \tag{11}$$

where $\mathbf{M}_2$ denotes the second central moment matrix,$n_\mathbf{A}$, $n_\mathbf{B}$, $n_\mathbf{C}$ are the respective cardinalities, and $\delta_{\mathbf{C},\mathbf{B}} = \boldsymbol{\mu}_\mathbf{C} - \boldsymbol{\mu}_\mathbf{B}$ is the difference between the partition means.

Let $\mathcal{P}$ denote the set of previously processed activations containing $n_{\mathcal{P}}$ samples, and let $\mathcal{N}$ denote an incoming new batch containing $n_{\mathcal{N}}$ samples. The combined set is given by $\mathcal{C} = \mathcal{P} \cup \mathcal{N}$, with a total of $n_C = n_{\mathcal{P}} + n_{\mathcal{N}}$ samples. When a new batch arrives, we first update the running mean of the activations using Eq. 11.

To update the sample covariance matrix, we define the weighting factor $\alpha = \frac{n_{\mathcal{P}}}{n_C}$, which implies $1 - \alpha = \frac{n_{\mathcal{N}}}{n_C}$. By decomposing the second moment across the disjoint sets $\mathcal{P}$ and $\mathcal{N}$, we derive the following online update formula for the covariance matrix:

$$\mathbf{C}_C = \alpha \mathbf{C}_{\mathcal{P}} + (1 - \alpha)\mathbf{C}_{\mathcal{N}} + \alpha(1 - \alpha)\boldsymbol{\delta}\boldsymbol{\delta}^T \tag{12}$$

where $\boldsymbol{\delta} = \boldsymbol{\mu}_{\mathcal{N}} - \boldsymbol{\mu}_{\mathcal{P}}$ represents the difference between the mean vectors of the new batch and the previously accumulated data. Using this recursive formulation, we can accurately construct the sample covariance matrix of each layer for an arbitrarily large calibration set without requiring the full activation history to be held in memory.

3.3. Denoising the Covariance Matrix with Marchenko-Pastur

3.3.1. Marchenko-Pastur Distribution

We observe that the spectral density of the activation covariance matrix $\mathbf{C}$ exhibits a bulk profile characteristic of the MP [26] distribution, strongly suggesting that the underlying signal subspace is corrupted by noise. To mitigate this, we denoise $\mathbf{C}$ by conducting eigendecomposition on $\mathbf{C}$ and by fitting its eigenvalue spectrum to the MP distribution. The MP distribution describes the asymptotic eigenvalue behavior of large random matrices. Specifically, for a large random matrix $\mathbf{X}$ of size $m \times n$ whose entries are independent and identically distributed (i.i.d.) with mean 0 and a variance of σ^2, the probability density function of the eigenvalues of its corresponding sample covariance $\mathbf{Y}_n = \frac{1}{n}\mathbf{X}\mathbf{X}^T$ is expressed as:

$$f(x) = \frac{\sqrt{(\lambda_+ - x)(x - \lambda_-)}}{2\pi\sigma^2\lambda x} \tag{13}$$

for $\lambda_- < x < \lambda_+$ and 0 otherwise. Here, $\lambda = \frac{m}{n}$ represents the aspect ratio of the matrix dimensions. According to the MP Law, if $\mathbf{X}$ satisfies the conditions, all of its non-zero eigenvalues will asymptotically fall within the theoretical bounds of λ_+ and λ_-. These bounds can be obtained by the following equation

$$\lambda_\pm = \sigma^2(1 \pm \sqrt{\lambda})^2. \tag{14}$$

3.3.2. Noise Variance of Noisy Matrices

Gavish and Donoho [13] focuses on the analysis of noisy matrices under the additive model $\mathbf{Y} = \mathbf{X} + \sigma\mathbf{Z}$, where $\mathbf{Y}$ is the observed data matrix, $\mathbf{X}$ is the underlying low-rank signal, and $\mathbf{Z}$ represents random noise. A primary challenge in practical application is that the true noise scale, σ, is typically unknown. To address this, the authors formulate a robust estimator $\hat{\sigma}(\mathbf{Y})$ using the ratio of the median singular value of the data matrix to the theoretical median of the MP distribution:

$$\hat{\sigma}(\mathbf{Y}) \stackrel{\text{def}}{=} \frac{y_{med}}{\sqrt{n\mu_\lambda}}, \tag{15}$$

where y_{med} is the median singular value of $\mathbf{Y}$. μ_λ is the median of MP distribution obtainable by solving for $\lambda_- < x < \lambda_+$

$$\int_{\lambda_-}^{x} \frac{\sqrt{(\lambda_+ - t)(t - \lambda_-)}}{2\pi t} dt = \frac{1}{2}. \tag{16}$$

Following our observation where eigenvalue distribution contains MP-like noise, visualized in Fig. 2. We assume the centered activation data matrix $\widetilde{\mathbf{X}}$ follows an additive noise model, $\widetilde{\mathbf{X}} = \widetilde{\mathbf{X}}_{\text{signal}} + \sigma\mathbf{Z}$, where the entries of the noise matrix $\mathbf{Z}$ adhere to the Marchenko-Pastur conditions.

Following Gavish and Donoho's work [13], we estimate the noise scale σ by comparing the median eigenvalue of $\mathbf{C}$ to the theoretical median of the MP distribution. The median singular value of the data matrix, denoted as s_{med}, relates to the median eigenvalue of the covariance matrix, Λ_{med}, obtained through eigendecomposition as follows:

$$s_{\text{med}} = \sqrt{N\Lambda_{\text{med}}}. \tag{17}$$

Substituting this relationship into the estimator Eq. 15 yields:

$$\hat{\sigma}(\widetilde{\mathbf{X}}) = \sqrt{\frac{\Lambda_{\text{med}}}{\mu_\lambda}} \tag{18}$$

where μ_λ is the theoretical median of the MP distribution.

To obtain μ_λ, we require matrix $\widetilde{\mathbf{X}}$'s aspect ratio λ. In this context, we define the aspect ratio as $\lambda = \frac{d}{N}$, where d is the number of input neurons and N is the calibration set size. We use calibration set size (number of independent images) rather than the total number of patches for the denominator because patches within a single image are highly correlated, using the total patch count would violate the i.i.d. noise assumption. By defining our sample size strictly as the calibration set size, we ensure the i.i.d. assumption holds. With the aspect ratio λ, we compute the theoretical median eigenvalue μ_λ via Eq. 16, which subsequently allows us to estimate the noise scale σ.

Using the estimated noise variance, we calculate the theoretical MP upper bound λ_+ using Eq. 14. After computing the eigendecomposition of the covariance matrix $\mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$, where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_d)$ contains the eigenvalues and $\mathbf{V} = [\mathbf{v}_1, \ldots, \mathbf{v}_d]$ the corresponding eigenvectors, we apply hard thresholding to retain only the signal components:

$$\mathbf{\Lambda}_{\text{signal}} = \text{diag}(\lambda_i) \quad \forall \lambda_i > \lambda_+ \tag{19}$$

$$\mathbf{V}_{\text{signal}} = [\mathbf{v}_i] \quad \forall \lambda_i > \lambda_+ \tag{20}$$

3.4. Denoised Variance Score

Using the retained signal components, we reconstruct the denoised covariance matrix:

$$\mathbf{C}_{\text{denoised}} = \mathbf{V}_{\text{signal}}\mathbf{\Lambda}_{\text{signal}}\mathbf{V}_{\text{signal}}^T \tag{21}$$

Finally, we extract the denoised variance scores for each neuron directly from the diagonal entries of the reconstructed matrix:

$$\mathbf{s}_{\text{dvar}} = \text{diag}(\mathbf{C}_{\text{denoised}}) \tag{22}$$

Empirically, we observe that the denoised variance scores $\mathbf{s}_{\text{dvar}}$ excel at high pruning ratios, whereas the raw variance scores $\mathbf{s}_{\text{var}} = \text{diag}(\mathbf{C})$, extracted from the original covariance matrix, remain highly effective at low pruning ratios. Thus, we store the raw scores prior to eigendecomposition and formulate a hybrid scoring metric, controlled by the target pruning ratio p:

$$\mathbf{s}_{\text{hybrid}} = (1 - p)\mathbf{s}_{\text{var}} + p\mathbf{s}_{\text{dvar}} \tag{23}$$

3.5. Pruning and Update Mechanism

Following the structural framework of Variance-Based Pruning (VBP), we target MLP blocks in Vision Transformers (ViTs) [6] and ConvNeXt [24] models. While we adopt this standard layer selection protocol, our approach introduces a novel neuron selection criterion and weight update mechanism.

Specifically, we globally rank the hidden dimensions based on our proposed hybrid score, and select the lowest-ranking p% of neurons across the entire network to consist the pruned indices set P. Rather than enforcing a uniform per-layer pruning ratio, the network dynamically allocates capacity, preserving critical layers while heavily penalizing redundant ones.

Table 1. Comparison of Top-1 accuracy (%) on ImageNet-1K across different pruning methods on DeiT and Swin. All models are evaluated immediately after compression without any post-pruning fine-tuning.

Model	Prune Ratio (%)	Top-1 Accuracy (%)					Model	Prune Ratio (%)	Top-1 Accuracy (%)				
		Full	Mag.	SNIP	VBP	Ours			Full	Mag.	SNIP	VBP	Ours
DeiT-T	40	72.16	14.64	55.80	57.56	**62.56**	Swin-T	40	81.38	47.39	37.99	64.25	**72.23**
	50		3.73	39.49	42.89	**56.40**		50		26.40	28.87	49.98	**66.12**
	60		0.81	23.59	26.97	**47.58**		60		7.91	17.87	26.03	**56.42**
DeiT-S	40	79.86	20.68	55.37	72.35	**73.71**	Swin-S	40	83.32	2.19	62.08	78.41	**80.69**
	50		4.24	50.65	66.03	**70.25**		50		0.22	45.36	69.91	**77.24**
	60		1.17	42.17	54.73	**65.25**		60		0.14	25.72	45.30	**70.79**
DeiT-B	40	81.98	1.95	65.03	76.49	**79.00**	Swin-B	40	85.27	35.62	57.02	78.70	**80.56**
	50		0.38	62.78	68.92	**75.85**		50		6.44	38.55	70.86	**76.16**
	60		0.16	52.82	50.63	**69.68**		60		1.10	23.81	55.88	**69.00**

Table 2. Comparison of parameter counts (M), MACs (G), and Top-1 accuracy (%) for DeiT models at a 50% pruning ratio.

Model	Parameters (M)		MACs (G)		Top-1 Accuracy (%)				
	Full	Pruned	Full	Pruned	Full	Mag.	SNIP	VBP	Ours
DeiT-T	5.72	3.94	1.26	0.91	72.16	3.73	39.49	42.89	**56.40**
DeiT-S	22.05	14.96	4.61	3.21	79.86	4.24	50.65	66.03	**70.25**
DeiT-B	86.57	58.24	17.59	12.01	81.98	0.38	62.78	68.92	**75.85**

To mitigate immediate performance degradation from neuron removal, we apply a two-stage compensation. First, we apply our novel OB^{2}C update to the outgoing weight matrix of the block (the second linear layer) by adding $\delta_{\mathbf{W}}$ computed by by the pruned indices set P to the weights $\mathbf{W}$. Then, we apply mean-shift compensation to the subsequent layer, which is the optimal bias term $\mathbf{b}^*$ added to the existing bias term.

After the two-stage compensation, we apply pruning. For a standard two-layer block, pruning reduces the intermediate hidden dimension from d_{hidden} to d_{pruned}. Consequently, the weight matrix of the first layer is reduced from $d_{\text{in}} \times d_{\text{hidden}}$ to $d_{\text{in}} \times d_{\text{pruned}}$, and the weight matrix of the second layer is reduced from $d_{\text{hidden}} \times d_{\text{out}}$ to $d_{\text{pruned}} \times d_{\text{out}}$. Because the external input (d_{in}) and output (d_{out}) dimensions remain strictly preserved, the pruned block seamlessly integrates back into the overarching network architecture.

4. Experiments

We evaluate our DVBP+OB^{2}C structured pruning framework on the Tiny, Small, and Base variants of the DeiT [32], Swin [23], and ConvNeXt [24] architectures, all pre-trained on the ImageNet-1K [5] dataset. To compute the activation covariance matrices, we construct a calibration dataset of 8,192 images, randomly sampled from the ImageNet-1K training split using a fixed seed. Notably, we omit data augmentation during calibration to accurately capture the true, unperturbed neuron activation statistics. Our evaluation primarily focuses on direct post-pruning accuracy retention, aiming to maximize off-the-shelf performance without relying on expensive retraining or fine-tuning pipelines. All experiments are conducted on a single NVIDIA GeForce RTX 3090 GPU. To guarantee a rigorous and fair comparison, we locally compute both the uncompressed baseline accuracies and the post-pruning performance of all evaluated methods under the exact same setup.

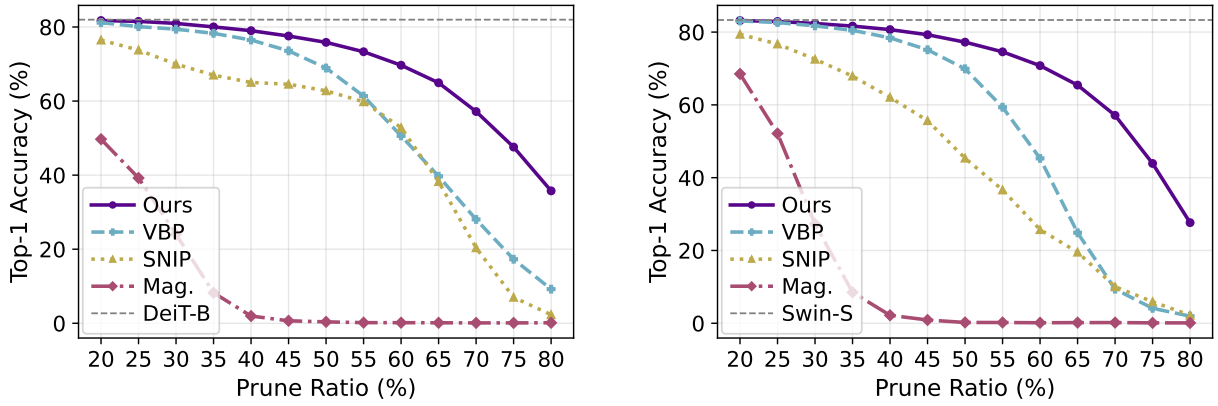
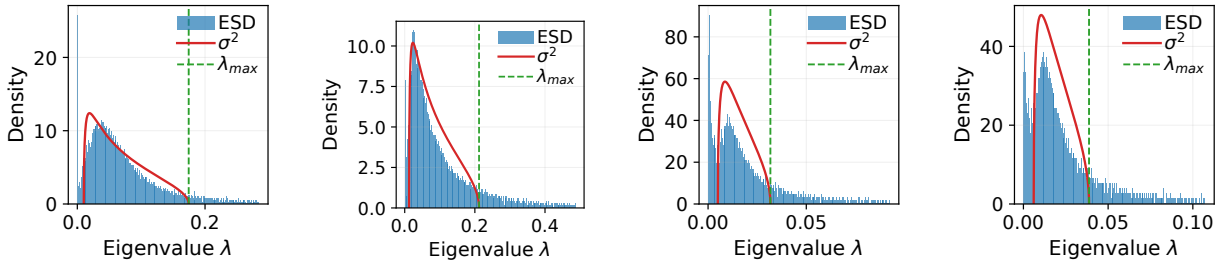


Figure 1. Pruning accuracy across diverse pruning ratios for DeiT-B (left) and Swin-S (right). SNIP and Magnitude pruning are adjusted to structured pruning.



(a) DeiT-B Layer 9 (b) DeiT-B Layer 10 (c) Swin-S Layer 11 (d) Swin-S Layer 12

Figure 2. Empirical spectral density fitted with the Marchenko-Pastur distribution. The largest 5% of eigenvalues are excluded for visual clarity.

4.1. Results

Our results demonstrate that the proposed approach consistently outperforms all baseline pruning methods across all evaluated models and pruning ratios. Notably, when compared to our closest competitor, VBP [2], the accuracy gap widens as the pruning ratio increases. For instance, on the Swin-S model, the post-pruning accuracy gap between our method and VBP is 2.28% at a 40% pruning ratio. This margin increases substantially to 7.33% at a 50% ratio, and expands even further to 25.49% at a 60% ratio.

As detailed in Table 1, at a 50% pruning ratio, our method successfully retains approximately 80% of the original accuracy for the Tiny models, and roughly 90% for the Small and Base models, representing a significant improvement over VBP. This diverging performance trend is further illustrated in Fig. 1, which shows the performance gap between our approach and VBP becoming increasingly pronounced from the 40% pruning threshold onward.

Finally, because our approach targets the same layers and utilizes the same fundamental pruning mechanism as the baselines, the overall parameter counts and multiply-accumulate operations (MACs) remain identical. However, despite this equivalent computational footprint, our method yields a vastly superior top-1 accuracy (Table 2).

As illustrated in Fig. 2, the empirical eigenvalue distributions of the models exhibit a noise structure characteristic of the Marchenko-Pastur (MP) [26] distribution. Furthermore, the fitted MP [26] curve accurately captures these noise components.

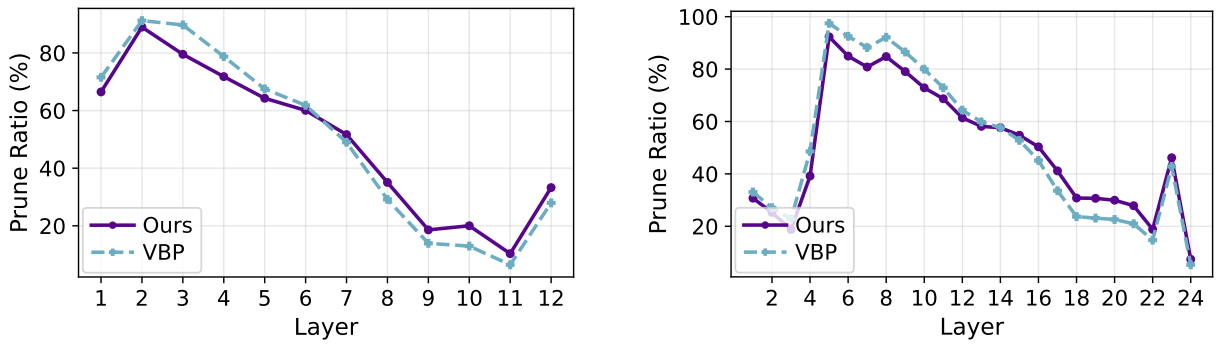


Figure 3. Layerwise pruning ratio for DeiT-B (left) and Swin-S (right), for model pruning rate 50%.

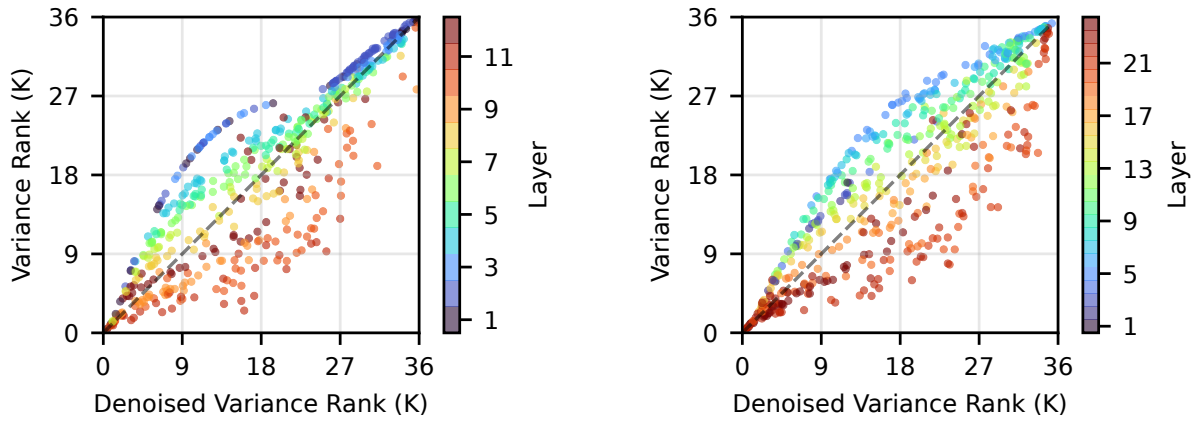


Figure 4. Spearman correlation plot of DeiT-B (left) and Swin-S (right) between denoised and raw variance score rank.

As shown in Fig. 3, our hybrid scoring mechanism yields a distinct pruning profile compared to VBP [2]. Specifically, our approach prunes the earlier layers less aggressively while increasing the pruning rate in the deeper layers. However, because both methods rely on a variance-based metric, their overall macroscopic trends remain similar. This relationship is further quantified by the Spearman rank correlation plot (Fig. 4), which reveals a general linear correlation between the ranks of the standard variance and our denoised scores, albeit with considerable deviations occurring within the middle portion of the rankings.

As demonstrated in Table 3 and Fig. 5, our pruning methodology generalizes effectively to the ConvNeXt [24] Tiny, Small, and Base models. SNIP [21] and Magnitude [14] pruning cause severe performance degradation on ConvNeXt architectures, reducing the accuracy to chance-level performance on the ImageNet-1K dataset. While VBP does achieve some accuracy retention, it is considerably lower than the baseline accuracies. Our method improves upon VBP by 29.46% for ConvNeXt-T, 21.88% for ConvNeXt-S, and 14.64% for ConvNeXt-B, representing a substantial improvement. This demonstrates that our method is highly applicable to diverse model architectures.

4.2. Analysis and Ablation Studies

Despite the significant accuracy gains, our method introduces minimal computational and memory overhead, ensuring its practical deployment on consumer-grade GPUs. Because we utilize a block-wise approach in OB^{2}C and precompute the pruning selections via our hybrid scoring mechanism, peak VRAM consumption is strictly bounded under 10 GB. Furthermore, the pruning process is highly time-efficient; as detailed in Table 4, while the execution time is approximately 2× that of VBP [2], it completes in under one minute

Table 3. Results of ConvNeXt Pruning at 50%.

Model	Top-1 Accuracy (%)				
	Full	SNIP	Mag.	VBP	Ours
ConvNeXt-T	84.17	1.90	0.10	15.44	**44.90**
ConvNeXt-S	85.17	1.76	0.10	28.13	**50.01**
ConvNeXt-B	85.81	29.91	0.13	56.12	**70.76**
Model	**Parameters (M)**				
	Full	Mag.	SNIP	VBP	Ours
ConvNeXt-T	28.58	16.98	15.94	12.61	12.96
ConvNeXt-S	50.22	27.32	28.50	23.37	24.04
ConvNeXt-B	88.59	48.80	46.92	41.18	42.63
Model	**MACs (G)**				
	Full	Mag.	SNIP	VBP	Ours
ConvNeXt-T	4.46	2.09	2.48	2.95	2.94
ConvNeXt-S	8.68	4.34	4.67	5.11	5.08
ConvNeXt-B	15.35	7.37	8.44	8.90	8.85

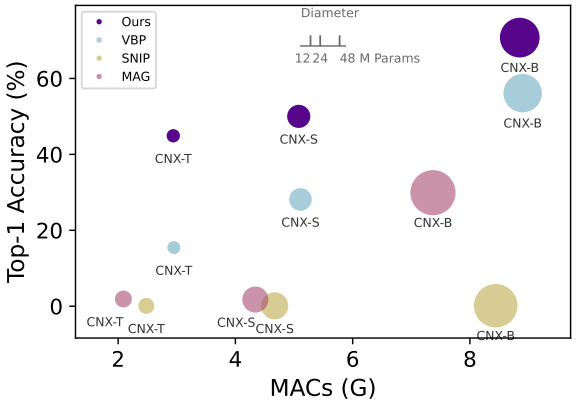


Figure 5. Overall Diagram for ConvNeXt pruning results.

Table 4. Memory and VRAM usage comparison between VBP.

Model	Peak VRAM (G)		Time (s)	
	VBP	Ours	VBP	Ours
DeiT-T	0.52	1.25	14.44	19.25
DeiT-S	0.98	2.48	13.16	21.79
DeiT-B	2.01	5.25	26.66	51.15
Swin-T	3.31	3.91	15.03	24.5
Swin-S	3.40	5.96	23.88	44.14
Swin-B	4.58	8.11	34.89	66.92

Table 5. Ablation study: denoising and OB^{2}C.

Model	Top-1 Accuracy (%)			
	Full	VBP	+D	+D +O
DeiT-T	72.16	42.90	44.32	**56.40**
DeiT-S	79.86	66.03	65.37	**70.25**
DeiT-B	81.98	68.92	69.90	**75.85**
Swin-T	81.38	49.98	52.48	**66.12**
Swin-S	83.32	69.91	71.21	**77.24**
Swin-B	85.27	70.86	70.11	**76.16**

for most architectures, requiring only 66.92 seconds for the larger Swin-B [23] model. These constrained resource requirements demonstrate that our approach can readily scale to larger models without necessitating specialized, high-memory hardware.

To evaluate the individual contributions of our proposed components, we analyze the isolated performance of the denoising mechanism and the OB^{2}C compensation framework, as detailed in Table 5. Integrating the denoised scoring yields marginal but consistent accuracy gains of 1-2% for the majority of the evaluated models, with the exception of DeiT-S [32] and Swin-B [23], which experience a negligible decrease of less than 1%. In contrast, the application of the OB^{2}C framework serves as the primary driver of performance recovery. Notably, it delivers a substantial absolute accuracy improvement of nearly 14% for DeiT-T [32], while consistently boosting the performance of the remaining models by 4-7%.

Because both VBP [2] and our proposed method can accommodate an arbitrary calibration set size, we investigate how varying this size impacts the final top-1 accuracy of each approach. While our primary experiments utilize a calibration dataset of 8,192 samples, our proposed method demonstrates robust scalability with respect to calibration size. As illustrated in Figure 6, increasing the calibration set initially yields top-1 accuracy improvements for DeiT-B pruning for both our method and VBP, with both approaches

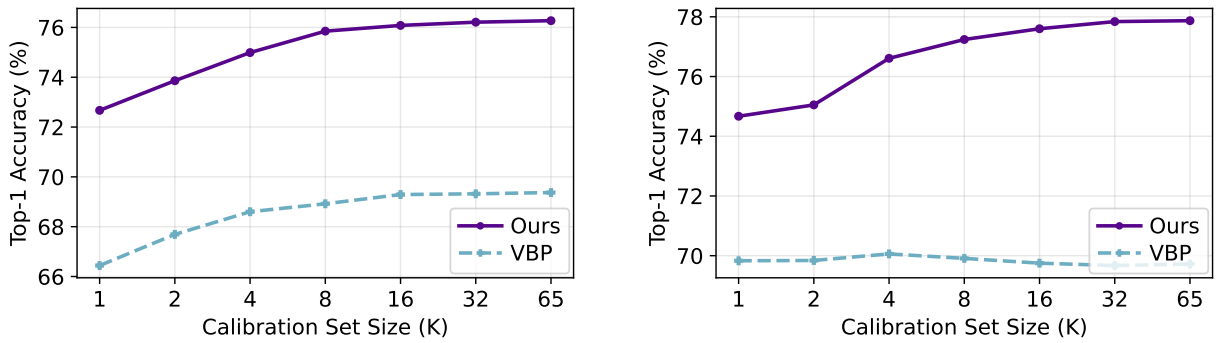


Figure 6. Top1-Accuracy (%) across various calibration set sizes for DeiT-B (left) and Swin-S (right).

Table 6. Ablation Study: comparing various denoising methods.

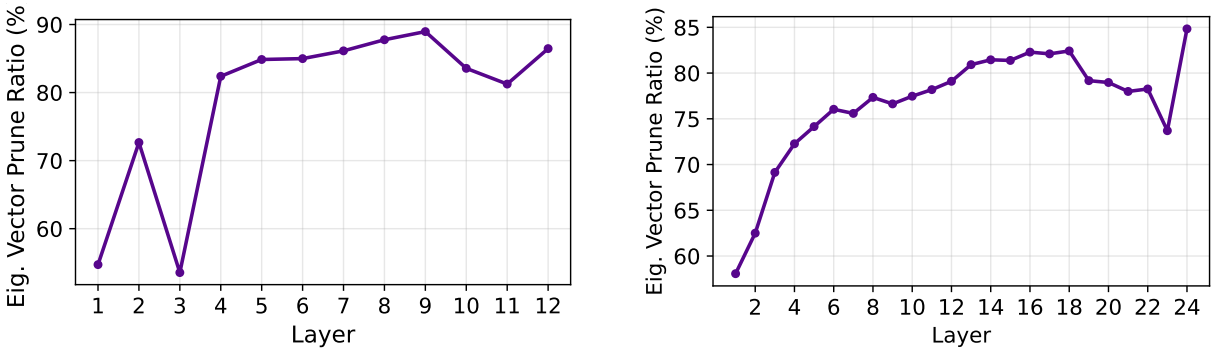
Model	Prune Ratio (%)	Top-1 Accuracy (%)						
		No Denoise	Top 50%	Top 60%	Top 70%	Top 80%	$<\lambda->$	$\lambda+>$
DeiT-B	50	75.76	**75.89**	75.71	75.73	75.76	0.14	75.85
	60	65.74	68.76	67.87	67.14	66.19	0.11	**69.68**
	70	45.49	51.81	49.84	47.49	45.89	0.12	**57.18**
Swin-S	50	76.67	**77.76**	77.64	77.41	76.95	0.28	77.24
	60	66.04	70.71	69.42	68.07	67.00	0.16	**70.79**
	70	37.26	48.89	43.30	40.65	38.12	0.12	**57.13**

reaching convergence at approximately 8K samples. However, for Swin-S [23], our method exhibits continuous performance gains as the calibration set expands, achieving convergence at roughly 32K samples. In stark contrast, the performance of VBP plateaus entirely in this scenario, showing no measurable improvement across a sweep from 1K to 65K samples.

We systematically compare various denoising strategies to validate our approach of fitting the MP [26] distribution and retaining only the signal eigenvectors associated with eigenvalues larger than λ_+. As demonstrated in Table 6, isolating eigenvectors with eigenvalues exceeding λ_+ consistently yields the highest top-1 accuracy among the evaluated denoising methods. Furthermore, this analysis confirms that eigenvectors with eigenvalues below λ_- predominantly capture noise, as reconstructing representations solely from these components degrades performance to chance-level accuracy. To quantify the extent of eigenvector pruning performed by our MP fitting method, we visualize the layer-wise pruning ratios in Figure 7. The results indicate that our method dynamically adjusts the pruning severity according to network depth; it discards approximately 60-70% of eigenvectors in the early layers and increases this ratio to roughly 80-90% in the deeper layers, a trend consistently observed in both the DeiT-B [32] and Swin-S [23] models. This adaptive behavior supports the hypothesis that representations become increasingly low-rank as they propagate deeper into the network, thereby requiring fewer signal eigenvectors to accurately capture the underlying information.

5. Conclusion

In this paper, we introduce DVBP + OB^{2}C, a novel structured pruning methodology that integrates statistical denoising techniques—specifically, Marchenko-Pastur (MP) distribution fitting within the VBP framework—with a highly efficient compensation mechanism. Our OB^{2}C framework advances the standard Optimal Brain Compression (OBC) approach by incorporating a multi-weight update strategy for pruned



(a) DeiT-B (b) Swin-S

Figure 7. Eigenvector prune ratio based on Marchenko-Pastur denoising.

weights, further refined by mean-shift compensation. Extensive evaluations demonstrate that our method achieves substantial improvements in post-pruning accuracy without the need for subsequent fine-tuning or retraining. Because this approach is fundamentally applicable to any linear layer, it generalizes seamlessly across a wide range of modern deep neural network architectures. We hope this work encourages further exploration into structured pruning within the computer vision community, paving the way for near-lossless compression combined with significant inference acceleration.

References

[1] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. In *NeurIPS*, 2024.

[2] Uranik Berisha, Jens Mehnert, and Alexandru Paul Condurache. Variance-Based Pruning for Accelerating and Compressing Trained Networks. In *ICCV*, 2025.

[3] Tony F. Chan, Gene H. Golub, and Randall J. LeVeque. Updating Formulae and a Pairwise Algorithm for Computing Sample Variances. In *COMPSTAT*, 1982.

[4] Arnav Chavan, Zhiqiang Shen, Zhuang Liu, Zechun Liu, Kwang-Ting Cheng, and Eric Xing. Vision transformer slimming: Multi-dimension searching in continuous optimization space. In *CVPR*, 2022.

[5] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *ICLR*, 2021.

[7] Murali Emani, Sam Foreman, Varuni Sastry, Zhen Xie, Siddhisatya Raskar, William Arnold, Rajeev Thakur, Venkatram Vishwanath, and Michael E. Papka. A Comprehensive Performance Study of Large Language Models on Novel AI Accelerators. *arXiv preprint arXiv:2310.04607*, 2023.

[8] Jonathan Frankle and Michael Carbin. The Lottery Ticket Hypothesis: Finding Sparse, Trainable Networks. In *ICLR*, 2019.

[9] Elias Frantar and Dan Alistarh. SparseGPT: Massive Language Models Can be Accurately Pruned in One-Shot. In *ICML*, 2023.

[10] Elias Frantar, Sidak Pal Singh, and Dan Alistarh. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning. In *NeurIPS*, 2022.

[11] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *ICLR*, 2023.

[12] Trevor Gale, Erich Elsen, and Sara Hooker. The State of Sparsity in Deep Neural Networks. *arXiv preprint arXiv:1902.09574*, 2019.

[13] Matan Gavish and David L. Donoho. The Optimal Hard Threshold for Singular Values is $4/\sqrt{3}$. *IEEE TIT*, 2014.

[14] Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both Weights and Connections for Efficient Neural Network. In *NeurIPS*, 2015.

[15] Babak Hassibi and David Stork. Second order derivatives for network pruning: Optimal Brain Surgeon. In *NeurIPS*, 1992.

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *CVPR*, 2016.

[17] Torsten Hoefler, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in Deep Learning: Pruning and growth for efficient inference and training in neural networks. *JMLR*, 2021.

[18] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3. In *ICCV*, 2019.

[19] Yann LeCun, John S. Denker, and Sara A. Solla. Optimal Brain Damage. In *NeurIPS*, 1989.

[20] Kwanhee Lee, Hyeondo Jang, Dongyeop Lee, Dan Alistarh, and Namhoon Lee. The Unseen Frontier: Pushing the Limits of LLM Sparsity with Surrogate-Free ADMM. In *ICLR*, 2025.

[21] Namhoon Lee, Thalaiyasingam Ajanthan, and Philip H. S. Torr. SNIP: Single-Shot Network Pruning based on Connection Sensitivity. In *ICLR*, 2019.

[22] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning Filters for Efficient ConvNets. In *ICLR*, 2017.

[23] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *ICCV*, 2021.

[24] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *CVPR*, 2022.

[25] Jian-Hao Luo and Jianxin Wu. An Entropy-based Pruning Method for CNN Compression. *arXiv preprint arXiv:1706.05791*, 2017.

[26] Vladimir A. Marchenko and Leonid Andreevich Pastur. Distribution of eigenvalues for some sets of random matrices. *Mathematics of the USSR-Sbornik*, 1967.

[27] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *ICLR*, 2017.

[28] Philippe Pebay. Formulas for Robust, One-Pass Parallel Computation of Covariances and Arbitrary-Order Statistical Moments. Technical report, Sandia National Laboratories, 2008.

[29] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

[30] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

[31] Hidenori Tanaka, Daniel Kunin, Daniel L. K. Yamins, and Surya Ganguli. Pruning neural networks without any data by iteratively conserving synaptic flow. In *NeurIPS*, 2020.

[32] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021.

[33] Xinchao Wang Xinyin Ma, Gongfan Fang. Llm-pruner: On the structural pruning of large language models. In *NeurIPS*, 2023.

[34] Huanrui Yang, Hongxu Yin, Maying Shen, Pavlo Molchanov, Hai Li, and Jan Kautz. Global Vision Transformer Pruning with Hessian-Aware Saliency. In *CVPR*, 2023.

[35] Lu Yu and Wei Xiang. X-Pruner: eXplainable Pruning for Vision Transformers. In *CVPR*, 2023.

[36] Junhan Zhu, Hesong Wang, Mingluo Su, Zefang Wang, and Huan Wang. OBS-Diff: Accurate Pruning for Diffusion Models in One-Shot. In *ICLR*, 2026.

A. Additional Details

In this supplementary material, the following additional details are provided:

- DVBP + OB^{2}C Algorithm
- Mathematical Proof of OB^{2}C
- Calibration Set Experiments
- Additional Experiments

A.1. DVBP + OB^{2}C Algorithm

In this section, we provide the step-by-step pseudocode for our combined DVBP and OB^{2}C methodology.

Algorithm 1 DVBP Pruning with OB^{2}C Compensation

Input: Pretrained model, calibration dataset $\mathcal{D}$, pruning ratio p

Compute layer-wise centered covariance matrices using Algorithm 2
Denoise covariance matrices and compute hybrid scores using Algorithm 3
Prune neurons and update weights using Algorithm 4
return Pruned model

Algorithm 2 Online Covariance Update per MLP Layer

Input: Calibration dataset $\mathcal{D}$, pretrained model

Initialize covariance matrix $\mathbf{C} \leftarrow 0$, mean vector $\boldsymbol{\mu} \leftarrow 0$, total samples $n \leftarrow 0$
for $i = 1 \ldots$ total batches **do**
    Obtain batch activations $\mathbf{x}$
    Compute batch mean $\boldsymbol{\mu}_{\mathcal{N}}$ and batch size $n_{\mathcal{N}}$
    $\widetilde{\mathbf{x}} \leftarrow \mathbf{x} - \boldsymbol{\mu}_{\mathcal{N}}$
    $\mathbf{C}_{\mathcal{N}} \leftarrow \frac{\widetilde{\mathbf{x}}^T \widetilde{\mathbf{x}}}{n_{\mathcal{N}}}$
    if $n == 0$ **then**
        $\mathbf{C} \leftarrow \mathbf{C}_{\mathcal{N}}$
        $\boldsymbol{\mu} \leftarrow \boldsymbol{\mu}_{\mathcal{N}}$
        $n \leftarrow n_{\mathcal{N}}$
    else
        $n_{\mathcal{P}} \leftarrow n$
        $\boldsymbol{\mu}_{\mathcal{P}} \leftarrow \boldsymbol{\mu}$
        $n \leftarrow n_{\mathcal{P}} + n_{\mathcal{N}}$
        $\alpha \leftarrow \frac{n_{\mathcal{P}}}{n}$
        $\boldsymbol{\delta} \leftarrow \boldsymbol{\mu}_{\mathcal{N}} - \boldsymbol{\mu}_{\mathcal{P}}$
        $\mathbf{C} \leftarrow \alpha\mathbf{C} + (1-\alpha)\mathbf{C}_{\mathcal{N}} + \alpha(1-\alpha)\boldsymbol{\delta}\boldsymbol{\delta}^T$
        $\boldsymbol{\mu} \leftarrow \boldsymbol{\mu}_{\mathcal{P}} + (1-\alpha)\boldsymbol{\delta}$
    end if
end for
return $\mathbf{C}, \boldsymbol{\mu}$

Algorithm 3 Denoise Covariance Matrix and Obtain Hybrid Scores

Input: Covariance matrix $\mathbf{C}$, mean vector $\boldsymbol{\mu}$, total samples n

$\mathbf{s}_{\text{var}} \leftarrow \text{diag}(\mathbf{C})$

$[\mathbf{V}, \mathbf{\Lambda}] \leftarrow \text{eig}(\mathbf{C})$
$d \leftarrow$ number of neurons
$\gamma \leftarrow \frac{d}{n}$
$\lambda_{\pm} \leftarrow (1 \pm \sqrt{\lambda})^2$
Obtain μ_λ by solving

$$\int_{\lambda_-}^{x} \frac{\sqrt{(\lambda_+ - t)(t - \lambda_-)}}{2\pi t} dt = \frac{1}{2}$$

$\lambda_{\text{med}} \leftarrow$ median eigenvalue of $\mathbf{\Lambda}$
$\hat{\sigma} \leftarrow \sqrt{\frac{\lambda_{\text{med}}}{\mu_\lambda}}$
$\lambda_+ \leftarrow \hat{\sigma}^2(1 + \sqrt{\lambda})^2$
$\mathbf{\Lambda}_{\text{signal}} \leftarrow \text{diag}(\lambda_i)$ for all $\lambda_i > \lambda_+$
$\mathbf{V}_{\text{signal}} \leftarrow [\mathbf{v}_i]$ for all $\lambda_i > \lambda_+$
$\mathbf{C}_{\text{denoised}} \leftarrow \mathbf{V}_{\text{signal}} \mathbf{\Lambda}_{\text{signal}} \mathbf{V}_{\text{signal}}^T$
$\mathbf{s}_{\text{dvar}} \leftarrow \text{diag}(\mathbf{C}_{\text{denoised}})$
$\mathbf{s}_{\text{hybrid}} \leftarrow (1 - p)\mathbf{s}_{\text{var}} + p\mathbf{s}_{\text{dvar}}$
$\mathbf{C}_{\text{denoised}} \leftarrow \mathbf{C}_{\text{denoised}} + \left(\delta \frac{\text{Tr}(\mathbf{C}_{\text{denoised}})}{d}\right) I$
$\mathbf{C}^{-1} \leftarrow \mathbf{C}_{\text{denoised}}^{-1}$
return $\mathbf{C}^{-1}, \mathbf{s}_{\text{hybrid}}$

Algorithm 4 DVBP Pruning with OB^{2}C Compensation

Input: Hybrid scores $\mathbf{s}_{\text{hybrid}}$, inverse covariance $\mathbf{C}^{-1}$, mean $\boldsymbol{\mu}$, pruning ratio p
Concatenate all block scores to obtain global scores $\mathbf{s}_{\text{global}}$
$\pi \leftarrow \text{argsort}(\mathbf{s}_{\text{global}})$
$k \leftarrow p \cdot |\mathbf{s}_{\text{global}}|$
$\mathcal{P}_{\text{global}} \leftarrow \{\pi_1, \ldots, \pi_k\}$
for each MLP block l **do**
    Map global indices to block indices: $\mathcal{P}_l$
    $\mathcal{K}_l \leftarrow \{1, \ldots, d_l\} \setminus \mathcal{P}_l$
    $\mathbf{W1}_{\text{pruned}} \leftarrow \mathbf{W1}_{\mathcal{K}_l,:}$
    $\mathbf{b1}_{\text{pruned}} \leftarrow \mathbf{b1}_{\mathcal{K}_l}$
    $\mathbf{W2}_{\text{new}}, \mathbf{b2}_{\text{new}} \leftarrow$ OB^{2}C update $(\mathbf{W2}, \mathbf{b2}, \mathbf{C}^{-1}, \boldsymbol{\mu}, \mathcal{P}_l)$
    $\mathbf{W2}_{\text{pruned}} \leftarrow [\mathbf{W2}_{\text{new}}]_{:,\mathcal{K}_l}$
end for

Algorithm 5 OB^{2}C Update

Input: Weights $\mathbf{W}$, bias $\mathbf{b}$, inverse covariance $\mathbf{C}^{-1}$, mean $\boldsymbol{\mu}$, pruned indices $\mathcal{P}$
$\mathbf{A} \leftarrow [\mathbf{C}^{-1}]_{\mathcal{P},\mathcal{P}}$
$\mathbf{B} \leftarrow [\mathbf{C}^{-1}]_{\mathcal{P},:}$
$\mathbf{W}_{\mathcal{P}} \leftarrow \mathbf{W}_{:,\mathcal{P}}$
$\Delta\mathbf{W} \leftarrow -\mathbf{W}_{\mathcal{P}}\mathbf{A}^{-1}\mathbf{B}$
$\mathbf{W}_{\text{new}} \leftarrow \mathbf{W} + \Delta\mathbf{W}$
$\mathbf{b}_{\text{new}} \leftarrow \mathbf{b} - \Delta\mathbf{W}\boldsymbol{\mu}$
return $\mathbf{W}_{\text{new}}, \mathbf{b}_{\text{new}}$

A.2. Mathematical Proof of OB^{2}C

In this section, we introduce the bias term into the layer-wise objective function as follows:

$$\arg\min_{\hat{\mathbf{W}},\mathbf{b}} \left\|\mathbf{W}\mathbf{X} - (\hat{\mathbf{W}}\mathbf{X} + \mathbf{b}\mathbf{1}^T)\right\|_F^2. \tag{24}$$

To solve this joint optimization problem, we first derive the optimal bias $\mathbf{b}^*$ for any given weight perturbation, which allows us to substitute it back into the objective and optimize solely for $\hat{\mathbf{W}}$.

Let $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ be the weight matrix of a single layer and $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times N}$ be the input activation data, where N is the calibration set size. The reconstruction error induced by pruning is:

$$E = \left\|\mathbf{W}\mathbf{X} - (\hat{\mathbf{W}}\mathbf{X} + \mathbf{b}\mathbf{1}^T)\right\|_F^2. \tag{25}$$

To find the optimal bias term, we take the derivative of E with respect to the vector $\mathbf{b}$ and set it to zero:

$$\frac{\partial E}{\partial \mathbf{b}} = -2\sum_{i=1}^{N}(\mathbf{W}\mathbf{x}_i - \widehat{\mathbf{W}}\mathbf{x}_i - \mathbf{b}) = \mathbf{0}. \tag{26}$$

This can be rewritten as:

$$\sum_{i=1}^{N}(\mathbf{W}\mathbf{x}_i - \widehat{\mathbf{W}}\mathbf{x}_i) - N\mathbf{b} = \mathbf{0}. \tag{27}$$

Solving for $\mathbf{b}$ yields the optimal bias vector $\mathbf{b}^*$:

$$\mathbf{b}^* = (\mathbf{W} - \widehat{\mathbf{W}})\frac{1}{N}\sum_{i=1}^{N}\mathbf{x}_i = (\mathbf{W} - \widehat{\mathbf{W}})\boldsymbol{\mu} = \Delta\mathbf{W}\boldsymbol{\mu}. \tag{28}$$

Substituting this optimal bias $\mathbf{b}^*$ back into the original error term E:

$$\begin{aligned}
E &= \left\|\mathbf{W}\mathbf{X} - (\widehat{\mathbf{W}}\mathbf{X} + \mathbf{b}^*\mathbf{1}^T)\right\|_F^2 \\
&= \left\|\mathbf{W}\mathbf{X} - (\widehat{\mathbf{W}}\mathbf{X} + (\mathbf{W} - \widehat{\mathbf{W}})\boldsymbol{\mu}\mathbf{1}^T)\right\|_F^2 \\
&= \left\|\mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}\mathbf{X} - (\mathbf{W} - \widehat{\mathbf{W}})\boldsymbol{\mu}\mathbf{1}^T\right\|_F^2 \\
&= \left\|(\mathbf{W} - \widehat{\mathbf{W}})\mathbf{X} - (\mathbf{W} - \widehat{\mathbf{W}})\boldsymbol{\mu}\mathbf{1}^T\right\|_F^2 \\
&= \left\|(\mathbf{W} - \widehat{\mathbf{W}})(\mathbf{X} - \boldsymbol{\mu}\mathbf{1}^T)\right\|_F^2 \\
&= \left\|\Delta\mathbf{W}\widetilde{\mathbf{X}}\right\|_F^2
\end{aligned} \tag{29}$$

where $\widetilde{\mathbf{X}} = \mathbf{X} - \boldsymbol{\mu}\mathbf{1}^T$ represents the mean-centered activations. Hence, the joint objective formula simplifies entirely to:

$$\arg\min_{\Delta\mathbf{W}} \left\|\Delta\mathbf{W}\widetilde{\mathbf{X}}\right\|_F^2. \tag{30}$$

Because the Frobenius norm separates across matrix rows, the second derivative of the simplified objective with respect to the full matrix $\Delta\mathbf{W}$ produces a block-diagonal Hessian:

$$\mathbf{H}_{\text{full}} = \mathbf{I}_{d_{\text{out}}} \otimes 2\widetilde{\mathbf{X}}\widetilde{\mathbf{X}}^T = \mathbf{I}_{d\text{out}} \otimes \mathbf{H}, \tag{31}$$

where $\mathbf{H} = 2\widetilde{\mathbf{X}}\widetilde{\mathbf{X}}^T$ is the shared row-wise Hessian. The inverse of a block-diagonal matrix is obtained by inverting each block:

$$\mathbf{H}_{\text{full}}^{-1} = \mathbf{I}_{d_{\text{out}}} \otimes \mathbf{H}^{-1}. \tag{32}$$

Thus, each row update depends only on the same $d_{\text{in}} \times d_{\text{in}}$ matrix $\mathbf{H}^{-1}$.

Substituting the unscaled covariance of the centered activations, $\mathbf{C} = \frac{1}{N}\widetilde{\mathbf{X}}\widetilde{\mathbf{X}}^T$, gives

$$\mathbf{H} = 2\widetilde{\mathbf{X}}\widetilde{\mathbf{X}}^T = 2N\mathbf{C}. \tag{33}$$

In the algorithm, the constant factor $2N$ cancels during the inverse update, allowing the exact row-wise Hessian to be replaced with the computationally efficient online covariance matrix $\mathbf{C}$.

A.3. Additional Experiments

A.3.1. Experimental Setting

All results rely on ImageNet-1K [5] training data for calibration, except for the LLM experiments in Table 13, which use Wikitext-2 [27]. Unless otherwise specified, the calibration set size is 8192. All experiments were conducted on a single NVIDIA GeForce RTX 3090 GPU.

A.3.2. Full Experimental Results

This section details the complete experimental results referenced in the main paper. Table 7 present the comprehensive tabular data for the calibration sweeps. Table 8 details full results of layerwise pruning ratio after our framework is applied. Table 9 provides whole eigenvector pruning ratio.

Furthermore, we provide extended visualizations: Figure 8 plots the full spearman correlation plots of denoised and raw variance scores, while Figure 9, Figure 10, and Figure 11 illustrate the complete empirical spectral density plots for both the DeiT-B and Swin-S models.

Table 7. Calibration set size experiments. Model pruning ratio is 50% for all models.

Model	Full	Method	Calibration Set Size						
			1024	2048	4096	8192	16384	32768	65536
DeiT-T	72.16	Ours	53.53	54.79	55.64	56.40	56.95	57.29	57.60
		VBP	43.65	42.98	42.98	42.89	42.93	42.78	42.75
DeiT-S	79.86	Ours	68.62	68.47	69.58	70.25	70.70	70.91	71.05
		VBP	65.90	65.98	66.00	66.03	66.30	66.25	66.18
DeiT-B	81.98	Ours	72.67	73.86	74.99	75.85	76.08	76.21	76.27
		VBP	66.44	67.69	68.64	68.92	69.29	69.32	69.37
Swin-T	81.38	Ours	61.42	63.05	64.79	66.12	67.27	67.63	67.81
		VBP	49.83	49.15	49.96	49.98	50.12	49.66	49.75
Swin-S	83.32	Ours	74.67	75.05	76.61	77.24	77.60	77.84	77.87
		VBP	69.83	69.84	70.06	69.91	69.75	69.67	69.72
Swin-B	85.27	Ours	74.60	74.43	75.15	76.16	77.44	78.15	78.62
		VBP	69.86	70.22	70.71	70.86	71.13	71.29	71.40

Table 8. Layerwise pruning ratios. Model pruning ratio is 50% for all models.

Model	Layerwise Pruning Ratio (%)											
	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11	L12
DeiT-T	14.45	41.02	26.17	40.76	51.69	50.52	42.58	50.78	52.08	56.64	80.86	92.45
DeiT-S	26.17	33.85	37.24	53.78	50.91	43.95	34.64	35.22	46.68	59.96	85.09	92.51
DeiT-B	66.47	88.96	79.52	71.78	64.26	60.06	51.66	35.06	18.59	20.02	10.35	33.27
Swin-T	24.74	29.95	14.06	45.57	84.38	73.50	67.77	59.11	50.26	46.81	62.79	12.04

Model	Layerwise Pruning Ratio (%)											
	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11	L12
Swin-S	30.73	25.26	18.75	39.19	92.32	84.96	80.79	84.77	79.04	72.85	68.68	61.39
Swin-B	33.79	22.27	11.23	31.54	91.80	84.38	72.66	75.59	71.04	66.65	59.57	54.00

Model	Layerwise Pruning Ratio (%)											
	L13	L14	L15	L16	L17	L18	L19	L20	L21	L22	L23	L24
Swin-S	58.14	57.62	54.75	50.33	41.15	30.79	30.66	29.95	27.86	18.82	46.16	7.42
Swin-B	49.41	49.27	46.92	47.17	43.60	37.84	22.75	15.77	11.28	6.84	69.14	47.39

Table 9. Layerwise eigenvector pruning ratios based on Marchenko-Pastur denoising.

Model	Layerwise Eigenvector Pruning Ratio (%)											
	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11	L12
DeiT-T	64.19	67.06	69.66	70.44	70.96	72.40	72.79	71.35	69.14	71.48	68.36	71.09
DeiT-S	67.19	72.85	75.13	78.06	78.58	80.08	81.64	78.91	78.19	73.89	67.25	73.76
DeiT-B	54.72	72.66	53.55	82.39	84.86	84.99	86.13	87.76	88.96	83.56	81.25	86.46
Swin-T	58.85	63.54	69.79	72.40	77.28	78.65	79.88	80.92	80.53	80.60	87.86	86.36

Model	Layerwise Eigenvector Pruning Ratio (%)											
	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11	L12
Swin-S	58.07	62.50	69.14	72.27	74.15	76.04	75.59	77.34	76.63	77.47	78.19	79.10
Swin-B	59.38	65.23	71.29	73.83	78.17	80.27	79.15	79.93	80.66	80.52	80.32	81.64

Model	Layerwise Eigenvector Pruning Ratio (%)											
	L13	L14	L15	L16	L17	L18	L19	L20	L21	L22	L23	L24
Swin-S	80.92	81.45	81.38	82.29	82.10	82.42	79.17	78.97	77.99	78.26	73.70	84.83
Swin-B	82.47	83.30	84.08	84.91	84.42	85.06	83.64	82.91	81.84	82.76	84.23	88.28

A.3.3. Further Analysis

To rigorously validate our framework, we perform additional analysis and ablation studies. Table 10 provides comparison with additional baselines adapted for structured MLP pruning: Hessian-based NViT [34], gradient-based SynFlow [31], and training-based ViT-Slim [4]. Our method outperforms all baselines.

Table 10. Experiment with additional baselines. Model pruning ratio is 50% for all models.

Model	Top-1 Accuracy (%)				
	Full	NViT	SynFlow	ViT-Slim	Ours
DeiT-B	81.98	58.01	2.24	72.34	**75.85**
Swin-S	83.32	53.39	0.10	73.56	**77.24**

Table 11. Ablation study: denoising, mean-shift, and OB^{2}C.

PR (%)	D	MS	OB²C	Top-1 Accuracy (%)					
				DeiT-T	DeiT-S	DeiT-B	Swin-T	Swin-S	Swin-B
70	X	O	O	35.33	56.98	45.49	32.36	33.80	54.06
	O	X	O	9.60	41.61	12.76	8.31	0.47	10.33
	O	O	X	14.19	38.34	42.07	16.89	35.37	39.79
	O	O	O	35.64	56.84	57.18	42.16	57.13	58.25

Table 11 ablates the individual contributions of denoising, mean-shift, and OB^{2}C. Denoising results in better accuracy at high pruning ratio, while mean-shift and OB^{2}C increase accuracy across all pruning ratios. Table 12 demonstrates robustness to the calibration set distribution by measuring the standard deviation of Top-1 accuracy across three calibration set seeds. We see no meaningful deviation.

A.3.4. Scalability and Applicability

To validate scalability, Table 13 evaluates our framework on larger architectures, including ViT-L [30] and Qwen2.5 1.5B [29]. For Qwen's SwiGLU MLPs, we calibrate on 128 Wikitext-2 samples (2048-token length) and apply uniform layer-wise pruning based on our hybrid scores. Specifically, we prune the lowest-scoring down-projection input channels and their corresponding up and gate-projection output channels, followed by OB^{2}C updates and mean-shift compensation on the down-projection layer. On the Wikitext-2 test set, our method outperforms all baselines, including the LLM-specific LLM-Pruner [33].

Furthermore, Table 14 demonstrates our method's generalizability to Multi-Head Attention (MHA) layers in ViT models. By averaging our hybrid scores across all attention heads, we uniformly prune the lowest-scoring QKV output channels and their corresponding out-projection input channels. With the application of OB^{2}C updates and mean-shift compensation, our approach again outperforms all evaluated baselines.

Table 12. Top-1 accuracy standard deviation measured across three calibration sets, with random seeds 100, 200, and 300.

Prune Ratio (%)	DeiT-T	DeiT-S	DeiT-B	Swin-T	Swin-S	Swin-B
50	0.20	0.10	0.12	0.21	0.08	0.11

Table 13. LLM and larger vision model pruned and calibrated on Wikitext-2 (128 samples) and ImageNet-1K (8192 samples).

Prune Ratio (%)	Qwen 2.5 1.5B Wiki-2 PPL (↓)				ViT-L Top-1 Acc. (%)		
	Full	LLM Pruner	VBP	Ours	Full	VBP	Ours
40		488.60	15.69	**12.56**		83.42	**84.30**
50	8.19	9557.71	28.35	**16.66**	87.90	77.60	**81.72**
60		29978.11	58.75	**32.44**		59.71	**76.58**

Table 14. Top-1 acc. results for structured pruning of MHA layer. Model pruning ratio is 30% for all models.

Model	MHA Pruning Top-1 Accuracy (%)				
	Full	Mag.	SNIP	VBP	Ours
DeiT-B	81.98	69.01	68.43	67.94	**73.09**
Swin-S	83.32	47.10	71.08	65.67	**73.00**

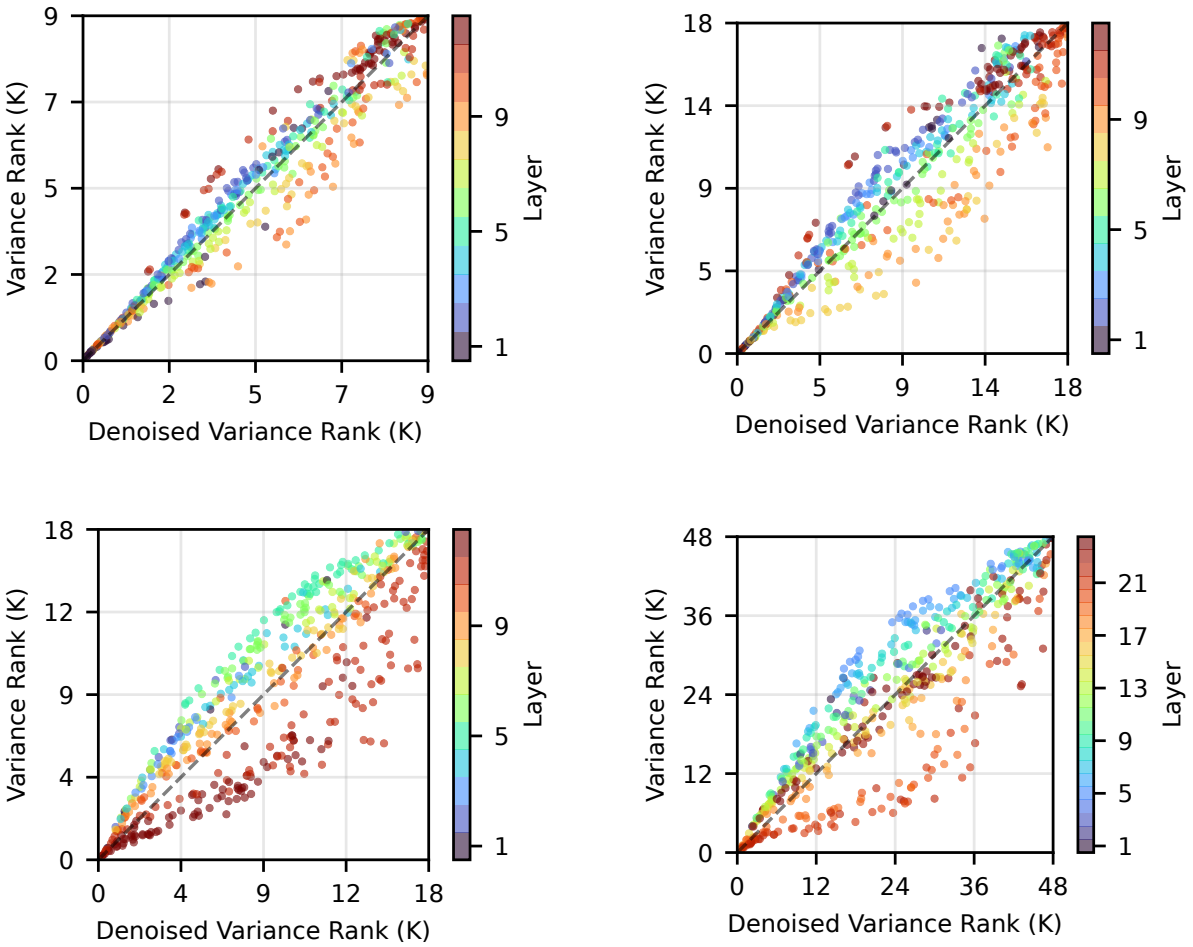
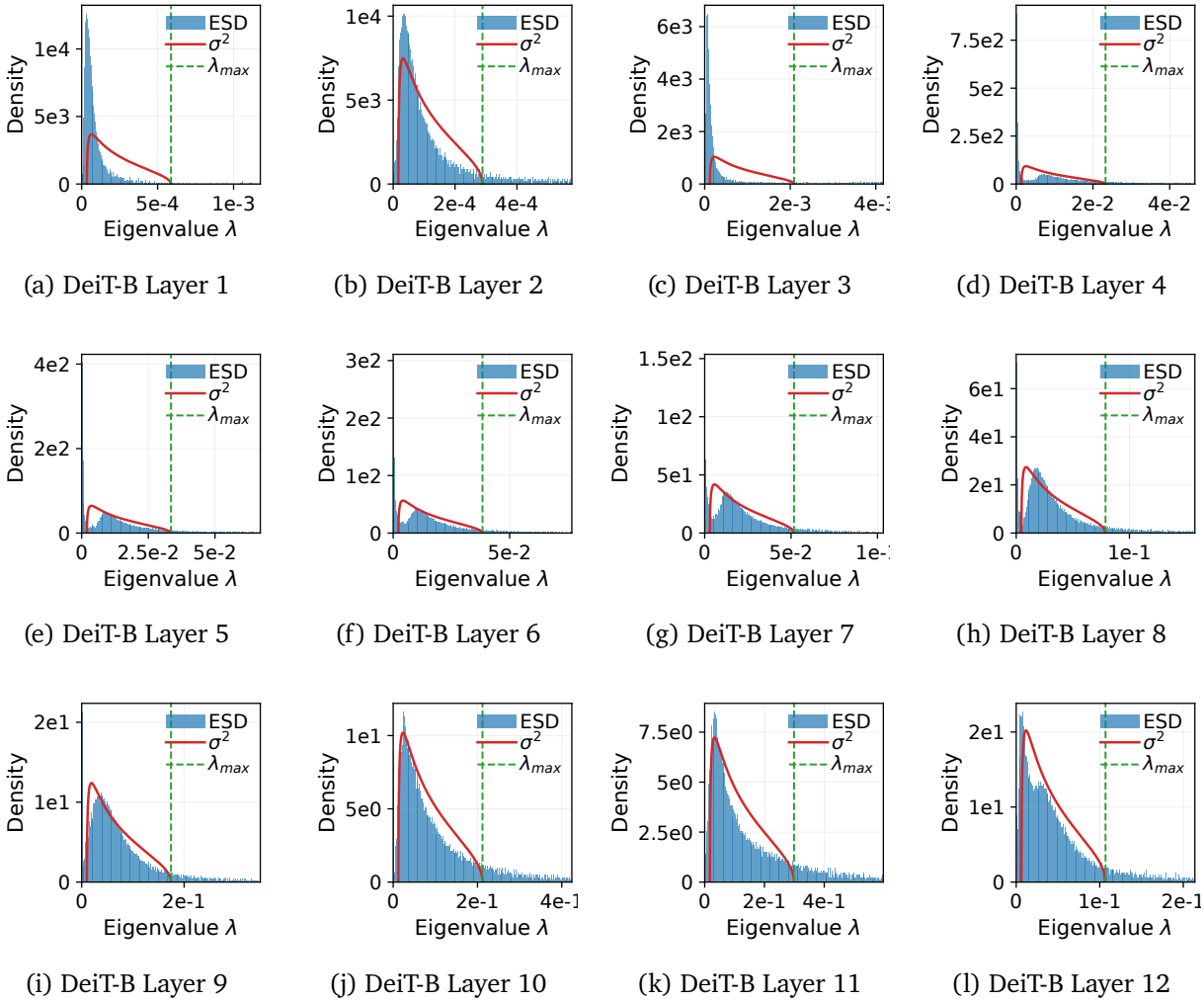


Figure 8. Spearman correlation plot of DeiT-T (top left), DeiT-S (top right), Swin-T (bottom left), Swin-B (bottom right) between denoised and raw variance score rank.

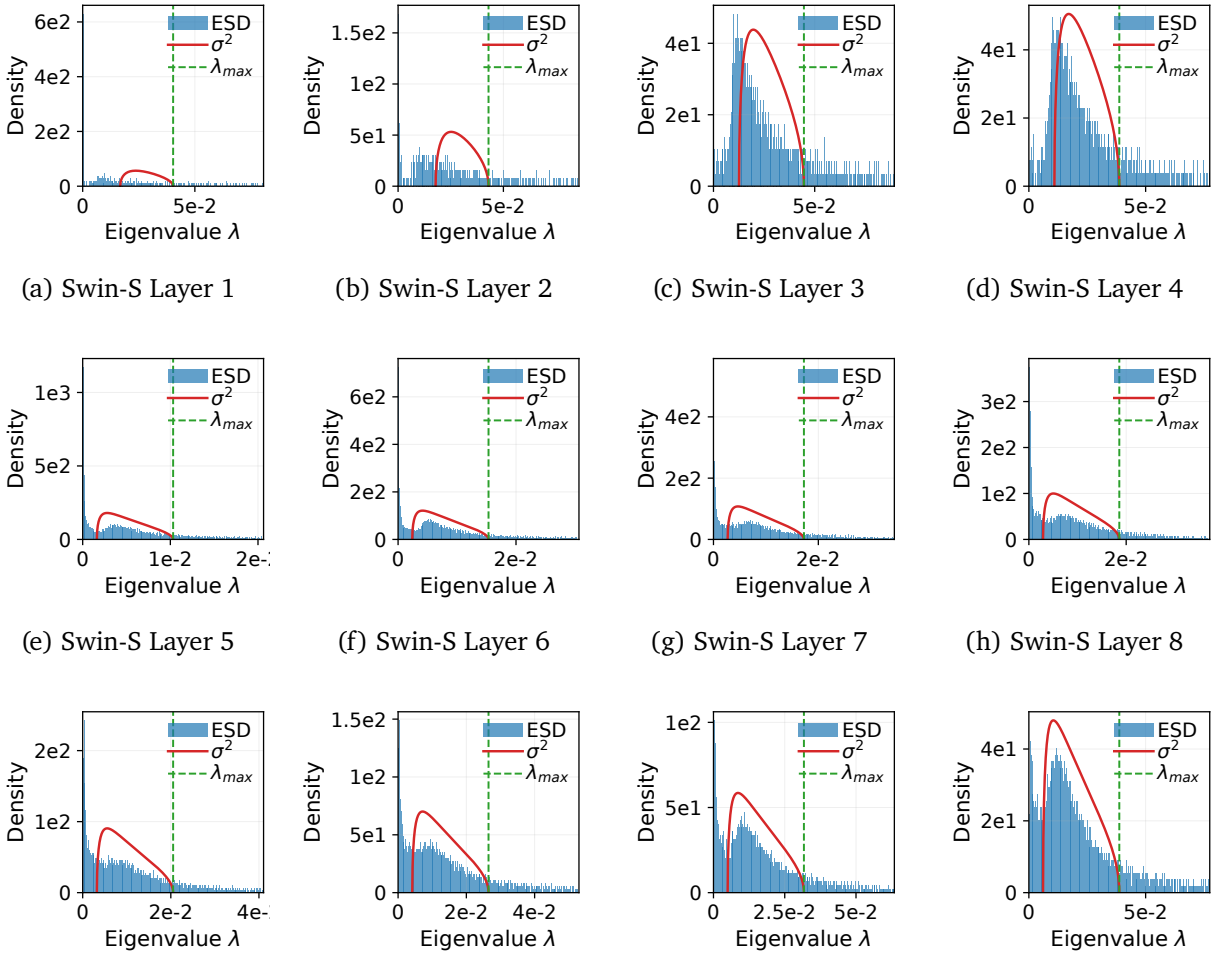


(a) DeiT-B Layer 1 (b) DeiT-B Layer 2 (c) DeiT-B Layer 3 (d) DeiT-B Layer 4

(e) DeiT-B Layer 5 (f) DeiT-B Layer 6 (g) DeiT-B Layer 7 (h) DeiT-B Layer 8

(i) DeiT-B Layer 9 (j) DeiT-B Layer 10 (k) DeiT-B Layer 11 (l) DeiT-B Layer 12

Figure 9. Empirical spectral density for all layers of DeiT-B fitted with the Marchenko-Pastur distribution.

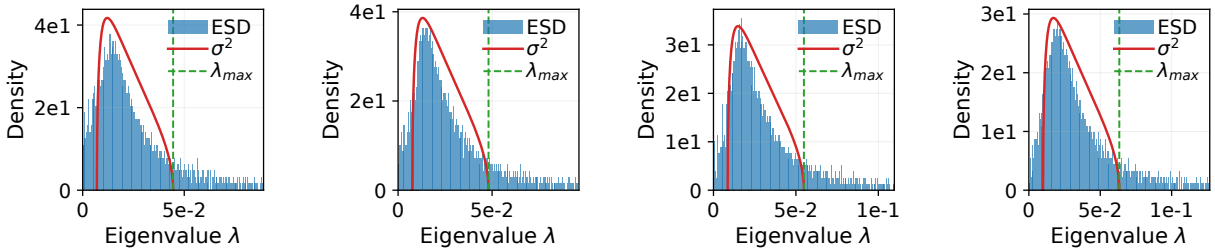


(a) Swin-S Layer 1 (b) Swin-S Layer 2 (c) Swin-S Layer 3 (d) Swin-S Layer 4

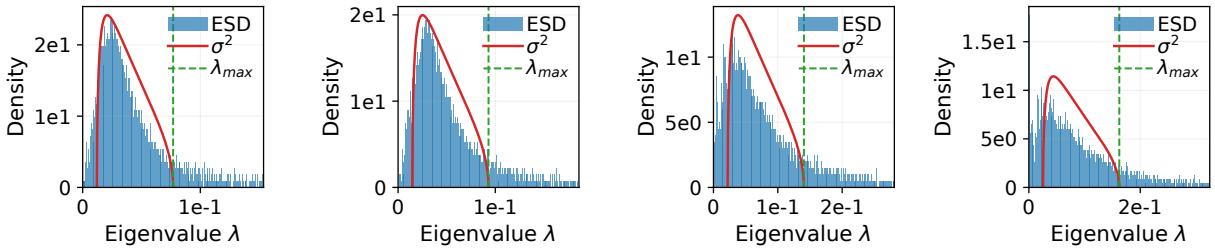
(e) Swin-S Layer 5 (f) Swin-S Layer 6 (g) Swin-S Layer 7 (h) Swin-S Layer 8

(i) Swin-S Layer 9 (j) Swin-S Layer 10 (k) Swin-S Layer 11 (l) Swin-S Layer 12

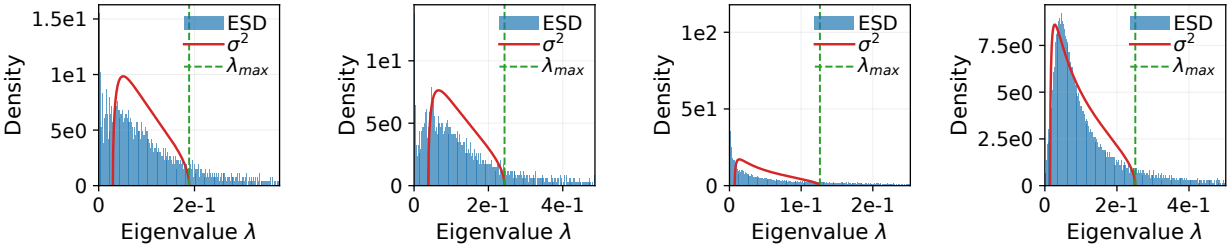
Figure 10. Empirical spectral density for layers 1-12 of Swin-S fitted with the Marchenko-Pastur distribution.



(m) Swin-S Layer 13 (n) Swin-S Layer 14 (o) Swin-S Layer 15 (p) Swin-S Layer 16



(q) Swin-S Layer 17 (r) Swin-S Layer 18 (s) Swin-S Layer 19 (t) Swin-S Layer 20



(u) Swin-S Layer 21 (v) Swin-S Layer 22 (w) Swin-S Layer 23 (x) Swin-S Layer 24

Figure 11. Empirical spectral density for layers 13-24 of Swin-S fitted with the Marchenko-Pastur distribution.